LATIHAN BAB 8

Nama : Khayla Qutsy Mauliddina
NIM : 25091397060
Kelas : 2025B
Mata Kuliah : Struktur Data

1. Analisis Kompleksitas Waktu

Operasi	Kompleksitas	Keterangan
Enqueue	O(1)	Penambahan elemen berlangsung cepat
Dequeue	O(1)/O(n)	Bergantung pada implementasi queue
isEmpty	O(1)	Pemeriksaan kondisi kosong bersifat konstan
Size	O(1)	Mengambil jumlah elemen secara langsung
Proses loop	O(n)	Dipengaruhi jumlah data yang diproses
Total sistem	O(n × k)	Kompleksitas keseluruhan pada kondisi terburuk

2. Bilangan dari 0 sampai 15 yang merupakan kelipatan 3 adalah:
   0, 3, 6, 9, 12, 15
   Setelah dimasukkan ke dalam queue, maka isi antrian menjadi:
   Front → [0, 3, 6, 9, 12, 15] ← Rear

3. Eksekusi Manual Kode
   Hasil akhir isi queue adalah:
   Front → [6, 9, 12, 15] ← Rear
   Proses perubahan queue secara bertahap:
   1. i = 0 → enqueue(0) → [0]
   2. i = 3 → enqueue(3) → [0, 3]
   3. i = 4 → dequeue() → [3]
   4. i = 6 → enqueue(6) → [3, 6]
   5. i = 8 → dequeue() → [6]
   6. i = 9 → enqueue(9) → [6, 9]
   7. i = 12 → enqueue(12) → [6, 9, 12]
   8. i = 15 → enqueue(15) → [6, 9, 12, 15]

4. Implementasi Program

Program dibuat menggunakan beberapa class yaitu Queue, Passenger, Agent, dan TicketCounterSimulation. Queue digunakan untuk menyimpan antrian penumpang, Passenger menyimpan waktu kedatangan, Agent bertugas melayani penumpang, dan TicketCounterSimulation mengatur jalannya simulasi.

Alur: Penumpang datang secara acak dan masuk ke dalam antrian. Agen akan melayani penumpang jika tersedia. Waktu tunggu dihitung dari selisih waktu pelayanan dan waktu kedatangan. Simulasi berjalan sesuai waktu yang ditentukan.

Code:

```
mport random

# ========================
# QUEUE
# ========================
class Queue:
    def __init__(self):
        self.data = []

    def isEmpty(self):
        return len(self.data) == 0

    def enqueue(self, item):
        self.data.append(item)

    def dequeue(self):
        if not self.isEmpty():
            return self.data.pop(0)

    def size(self):
        return len(self.data)


# ========================
# PASSENGER
# ========================
class Passenger:
    def __init__(self, time):
        self._timeArrived = time

    def timeArrived(self):
```

```
        return self._timeArrived


# ========================
# AGENT (PETUGAS)
# ========================
class Agent:
    def __init__(self):
        self._busy = False
        self._finishTime = 0

    def isFree(self):
        return not self._busy

    def startService(self, passenger, finishTime):
        self._busy = True
        self._finishTime = finishTime

    def isFinished(self, currentTime):
        return self._busy and currentTime >= self._finishTime

    def stopService(self):
        self._busy = False


# ========================
# SIMULATION
# ========================
class TicketCounterSimulation:
    def __init__(self, numAgents, numMinutes, arrivalProb, serviceTime):
        self._passengerQ = Queue()
        self._theAgents = [Agent() for _ in range(numAgents)]

        self._numMinutes = numMinutes
        self._arriveProb = arrivalProb
        self._serviceTime = serviceTime

        self._totalWaitTime = 0
        self._numPassengers = 0

    # ========================
    # PENUMPANG DATANG
    # ========================
    def _handleArrival(self, curTime):
        if random.random() <= self._arriveProb:
            passenger = Passenger(curTime)
```

```
        self._passengerQ.enqueue(passenger)
        self._numPassengers += 1

    # ========================
    # MULAI LAYANAN
    # ========================
    def _handleBeginService(self, curTime):
        for agent in self._theAgents:
            if agent.isFree() and not self._passengerQ.isEmpty():
                passenger = self._passengerQ.dequeue()
                agent.startService(passenger, curTime + self._serviceTime)

                waitTime = curTime - passenger.timeArrived()
                self._totalWaitTime += waitTime

    # ========================
    # SELESAI LAYANAN
    # ========================
    def _handleEndService(self, curTime):
        for agent in self._theAgents:
            if agent.isFinished(curTime):
                agent.stopService()

    # ========================
    # RUN SIMULASI
    # ========================
    def run(self):
        for currentTime in range(self._numMinutes):
            self._handleArrival(currentTime)
            self._handleEndService(currentTime)
            self._handleBeginService(currentTime)

        self._printResults()

    # ========================
    # HASIL
    # ========================
    def _printResults(self):
        print("=== HASIL SIMULASI ===")
        print("Total penumpang:", self._numPassengers)

        if self._numPassengers > 0:
            avgWait = self._totalWaitTime / self._numPassengers
            print("Rata-rata waktu tunggu:", round(avgWait, 2))
        else:
            print("Tidak ada penumpang")
```

```
        print("Sisa antrian:", self._passengerQ.size())


# ========================
# MAIN PROGRAM
# ========================
if __name__ == "__main__":
    sim = TicketCounterSimulation(
        numAgents=2,      # jumlah petugas
        numMinutes=20,    # waktu simulasi
        arrivalProb=0.3,  # peluang kedatangan
        serviceTime=3     # waktu pelayanan
    )

    sim.run()
```

5. Untuk mengubah satuan waktu dari menit ke detik, perlu dilakukan konversi pada beberapa parameter utama sebelum simulasi dijalankan.

   Langkah-langkah yang dilakukan:
   - Waktu total simulasi diubah menjadi detik → numMinutes × 60
   - Waktu pelayanan diubah → serviceTime × 60
   - Interval kedatangan diubah menjadi probabilitas per detik → 1.0 / (betweenTime × 60)

6. Fungsi reverse queue

```
from lliststack import Stack

def reverseQueue(queue):
    stack = Stack()

    # Memindahkan semua elemen queue ke stack
    while not queue.isEmpty():
        stack.push(queue.dequeue())

    # Mengembalikan elemen ke queue (urutan terbalik)
    while not stack.isEmpty():
```

```
    queue.enqueue(stack.pop())

return queue
```